TECHNICAL README

Code Optimization using Move Semantics and Dynamic Memory Allocation (DMA)

Project: SentimentStream – NLP Sentiment Scorer

Team Members:

Katyayani Verma (25BCE0612)

Pushpanvitha Pusuluri (25BCB0081)

Harshith M Singh (25BCE2900)

Hamsika Gokavarapu (25BDS0067)

GitHub Repository Link :

https://github.com/katyayani29verma-cyber/NLP-SENTIMENT-SCORER

Introduction :

The SentimentStream – NLP Sentiment Scorer project analyzes text input to determine the emotional tone using rule-based sentiment scoring techniques. The application processes large text datasets, which requires efficient memory management and optimized data transfer methods to maintain performance.

In Natural Language Processing (NLP) applications, text size can vary widely depending on the input source. Handling such data efficiently demands careful memory management and avoiding unnecessary data duplication.

To achieve better performance, the project combines two key C++ optimization techniques:

• Dynamic Memory Allocation (DMA)

• Move Semantics (C++11 feature)

These techniques improve:

• runtime efficiency

• memory use

• program scalability

• speed of text processing

The implementation shows practical uses of programming techniques that enhance the performance of applications dealing with large text data.

Role of Memory Optimization in NLP Applications :

Sentiment analysis applications process raw text data, which often involves:

• reading large input files

• dynamically storing tokens (words)

• transferring data between objects

• managing intermediate buffers

• performing frequent lookups

Using inefficient memory management can lead to:

• excessive memory use

• slower execution time

• unnecessary copying of large strings

• increased computational load

Therefore, effective memory strategies like DMA and Move Semantics are crucial for developing scalable NLP systems.

Dynamic Memory Allocation (DMA)

1) Concept of DMA:

Dynamic Memory Allocation refers to assigning memory at runtime rather than at compile time. This allows programs to request memory as needed and to release it after processing.

Unlike static arrays, which require fixed sizes, dynamic memory allocation offers flexibility for handling variable-sized data like text files.

DMA is implemented in C++ using:

new → allocates memory

delete → releases memory

This technique ensures effective memory use, especially when the exact size of input data is unknown in advance.

2) DMA Implementation in SentimentStream :

In this project, a custom Buffer class is used to temporarily hold the raw text data read from the input file before tokenization.

The Buffer class dynamically allocates memory based on the size of the text file.

Conceptual representation:

```
char* data = new char[textSize];
```

This ensures that:

• memory allocation matches input size

• no extra memory is reserved

• large input files can be processed effectively

After processing completes, memory is released with:

delete[] data;

This prevents memory leaks and ensures proper resource cleanup.

3) Why DMA is necessary in this project?

The input text file may vary in size based on the dataset used for sentiment analysis. Using fixed-size arrays would lead to:

• wasted memory if the array size is too large

• failure to process large files if the array size is too small

• poor resource use

DMA solves these problems by allowing flexible allocation based on actual data size.

4) Advantages of DMA in SentimentStream :

- Efficient memory use

Only necessary memory is allocated based on text size.

- Flexibility

The system can handle small or large text files without changes.

- Scalability

The program can manage large NLP datasets.

- Reduced memory waste

Prevents unnecessary memory reservations.

- Improved system performance

Effective memory use lowers system load.

Move Semantics

1) Concept of Move Semantics :

Move Semantics is a feature introduced in C++11 that allows resources to be transferred from one object to another without copying the actual data.

Traditionally, when passing objects like strings or vectors between functions or classes, a copy of the object is made. Copying large objects takes time and memory.

Move Semantics avoids this inefficiency by transferring ownership of the resource instead of duplicating it.

Move semantics is implemented using:

std::move()

Instead of copying the contents of an object, move semantics transfers the internal memory pointer to the new object.

2) Move Semantics Implementation in SentimentStream :

After reading the input text file, the text data must be moved to processing components like the Buffer class or tokenization module.

Without move semantics:

```
string textData = readFile();

Buffer buffer(textData);
```

Here, the entire string is copied into the buffer object.

With move semantics:

```
Buffer buffer(std::move(textData));
```

In this case:

• ownership of textData memory is transferred

• no duplicate copy of the string is made

• the original object becomes empty or inactive

• memory operations are reduced

3) *Why Move Semantics is important in NLP processing* :

Text data can be large, especially when dealing with paragraphs, articles, or datasets. Repeated copying of large strings can:

• increase execution time

• consume more RAM

• decrease performance efficiency

Move semantics prevents unnecessary duplication and ensures efficient transfer of large text resources.

4) Advantages of Move Semantics in SentimentStream :

- Faster execution

Avoids costly deep copying of strings.

- Reduced memory consumption

Only one instance of memory is maintained.

- Efficient object transfer

Large data objects are transferred quickly.

- Improved runtime performance

The program runs faster with large text input.

- Optimal resource management

Ensures minimal use of system memory.

Combined Optimization Strategy

Dynamic Memory Allocation and Move Semantics work together to greatly improve the efficiency of the SentimentStream system. DMA ensures flexible memory allocation according to text size. Move Semantics allows efficient transfer of allocated memory between objects. Together, they provide:

- effective memory management
- faster text processing
- reduced computational load
- scalable performance
- optimized handling of large datasets

Impact on System Performance

By using DMA and Move Semantics, the SentimentStream application achieves:

Improved Speed

Less copying of large strings speeds up execution.

Better Memory Use

Memory is allocated only when needed.

Reduced Overhead

Avoids repeated allocation and copying.

Scalability

The program can handle large text inputs without losing performance.

Reliability

Proper memory management prevents leaks and runtime crashes.

Technical Justification of Optimization Approach

NLP-based applications often deal with large unstructured text inputs. Poor memory management can lead to slow execution and high resource use.

Using DMA ensures that memory allocation adjusts to varying input sizes during runtime.

Using move semantics guarantees efficient transfer of large text data between modules like the file reader, buffer, tokenizer, and scorer components.

The combined use of these techniques shows a solid understanding of advanced C++ optimization practices and enhances the real-world relevance of the sentiment analysis system.

Conclusion

The SentimentStream project effectively applies Dynamic Memory Allocation and Move Semantics to enhance program performance and memory use.

DMA allows flexible memory allocation based on input size, ensuring effective use of system resources.

Move semantics reduces unnecessary duplication of large text data by transferring ownership of resources instead of copying them.

Together, these techniques significantly improve execution efficiency, cut down memory overhead, and enhance scalability in the sentiment analysis system.

The implementation illustrates a practical understanding of advanced C++ memory optimization concepts and their significance in developing efficient Natural Language Processing applications.